Mergeable SNARGs for Trapdoor Languages and Their Applications

Zvika Brakerski
Weizmann Institute

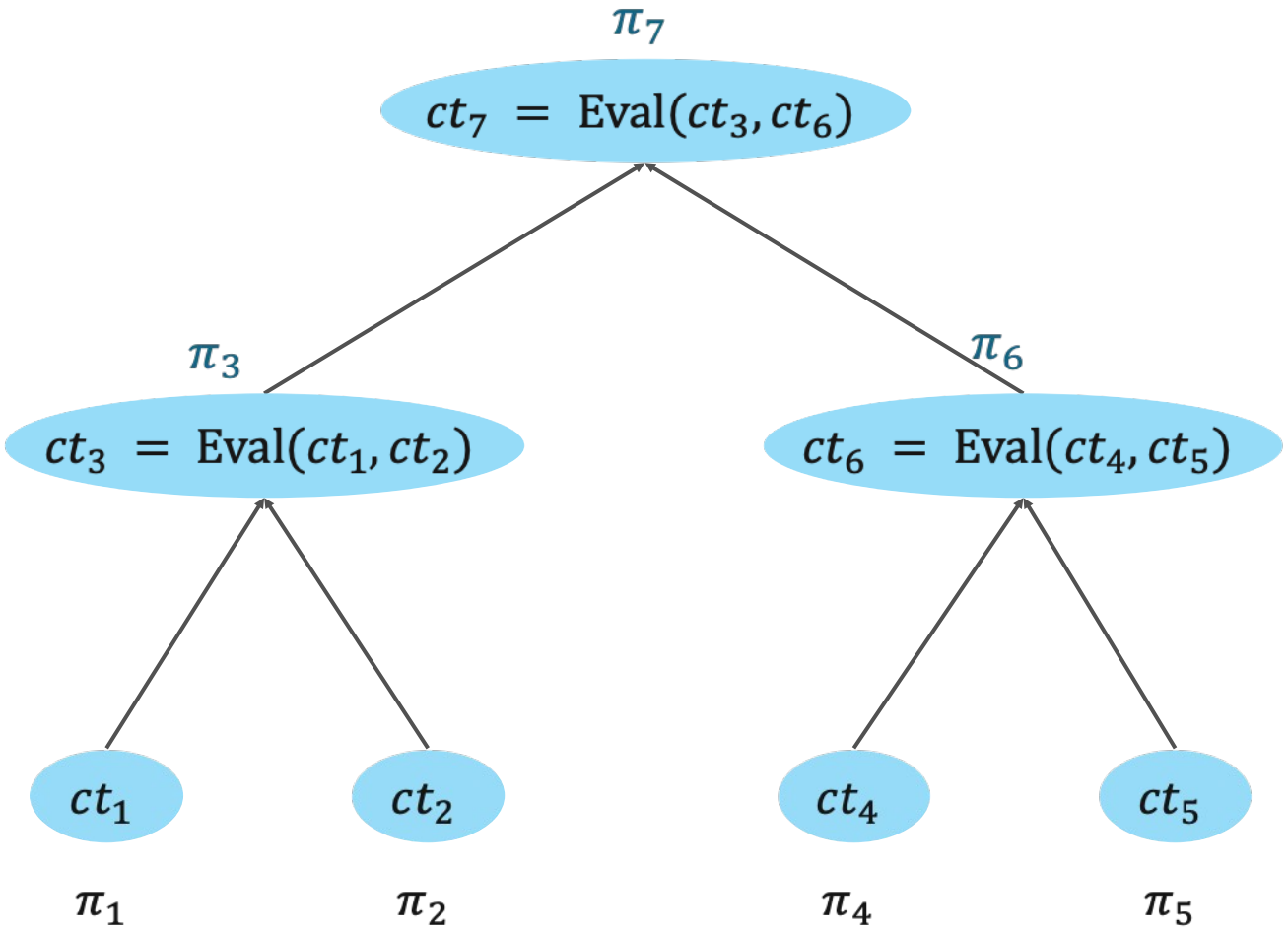
Maya Farber Brodsky
Tel Aviv University

Omer Paneth
Tel Aviv University

Tomer Solomon
Tel Aviv University

Motivating Example

- Well-formed ciphers (with low noise)
- Proof for cipher?
  - Proof for evaluated ciphers?
- **No concatenation?**



Mergeable Language

Language $\mathcal{L}$ with efficient **instance merger** (IM) algorithm such that:

- Syntax: $IM(x_0, x_1) = x$ is also an instance
- Soundness: $x_0, x_1 \in \mathcal{L} \Rightarrow IM(x_0, x_1) \in \mathcal{L}$

Example

$$\mathcal{L}_{\mathrm{FHE}}^{pk} = \{ct : ct \text{ is a well-formed FHE ciphertext under } pk\}$$

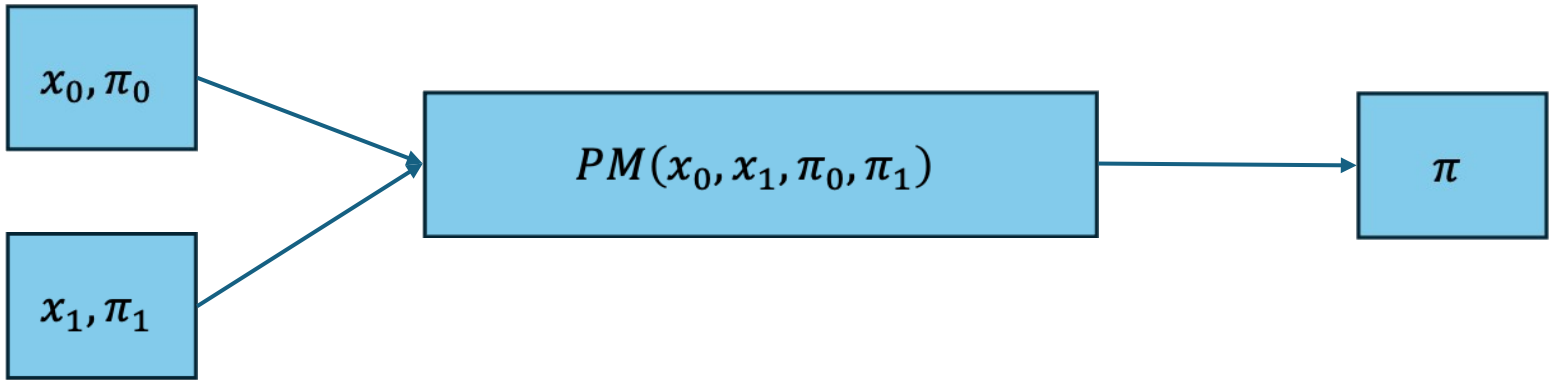
$$IM(ct_0, ct_1) = ct_0 \oplus ct_1$$

Mergeable Arguments

Let $\mathcal{L}$ be a language with an instance merger IM

A mergeable argument for $(\mathcal{L}, IM)$ consists of:

- A non-interactive argument (G, P, V) for $\mathcal{L}$
- proof merger PM



Merge Completeness and Efficiency

- Merge Completeness:

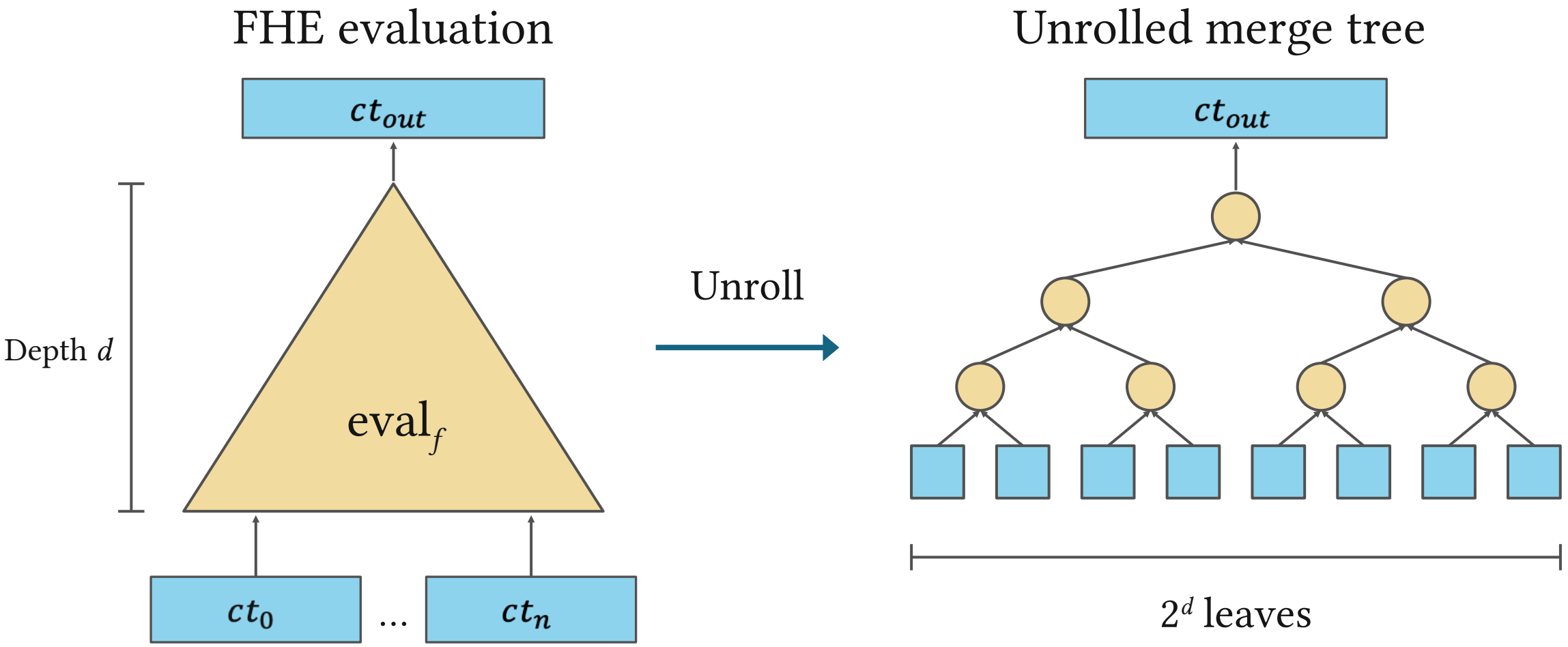
$$\forall x_0, x_1, \pi_0, \pi_1$$
$$x = IM(x_0, x_1)$$
$$\pi = PM(x_0, x_1, \pi_0, \pi_1)$$
$$V(x_0, \pi_0) = V(x_1, \pi_1) = 1 \Rightarrow V(x, \pi) = 1$$

- Merge Efficiency: $|\pi| \ll |\pi_0| + |\pi_1|$

Previous Work

	SNARK [CT10, BCCT13, CCGOS23, CGSY24,...]	**LWE** [PP26]
Language	NP	**P**
Proof size	$poly(\lambda)$	$\boldsymbol{poly(\lambda, d)}$
Security	**Constant-depth merge trees**	**Polynomial-size maximal merge trees**

The Maximal Merge Tree



Our Results

- Mergeable argument systems for NP languages with **trapdoor**
  - Security holds for unbounded merge tree

What's a Trapdoor Language?

- Family of NP languages $\mathcal{L} = \{\mathcal{L}_k\}$

- Each language $\mathcal{L}_k$ can be decided efficiently with a trapdoor τ_k
  - Computing τ_k may be hard from k
  - Honest parties know k but not τ_k

- Example: $\mathcal{L} = \{\mathcal{L}_{pk}\}$

$$\mathcal{L}_{pk} = \{ct : ct \text{ is a well-formed FHE ciphertext under } pk\}$$

Our Results

- Theorm 1: mergeable arguments for trapdoor languages from **LWE**
  - Overhead linear in merge-tree depth

- Theorm 2: mergeable arguments for trapdoor languages from **subexp IO + OWF**
  - Proof size is fixed $poly(\lambda)$

- **Both support arbitrary merge trees**
- Proof merge time scales with decision time of $\mathcal{L}_k$ given τ_k

Applications

- Multi-hop CCA1 FHE

- Adaptive multi-hop aggregate signatures

Application 1: Multi-hop CCA1 FHE

- First construction of multi-hop CCA1 FHE
  - LWE: ciphers grow with computation depth
  - Sub-exp IO: ciphers size does grow

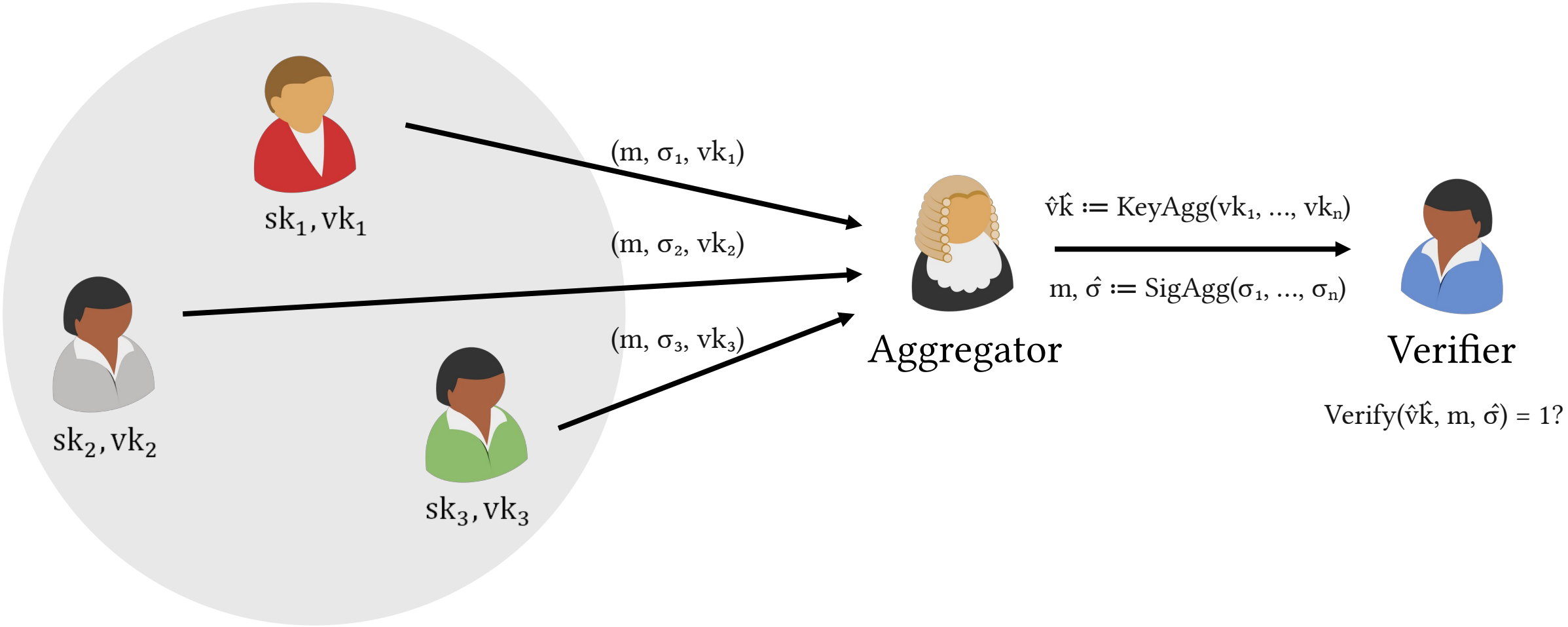
- Previous constructions achieve single-hop from:
  - SNARK [CRRV17, YKT17,...]
  - LWE [YYS25]

Go Beyond Single-Hop

- Idea: extend Naor-Yung to FHE with mergeable argument system for trapdoor languages
  - Trapdoor language: "ct_0 and ct_1 encrypt the same message"

- Merge proofs along homomorphic evaluation
  - Base: NIZK proof
  - Step: merge proofs of previous steps

Application 2: Adaptive Multi-hop Aggregate Signatures [IN83, BGLS03]



Signatures - Previous Works

- Single-hop construction from BARG [WW22]
- Multi-hop construction from Rate-1 BARGs [DGKV22]

- **Limitation:** The aggregation graph is **static**, i.e. cannot be chosen adaptively

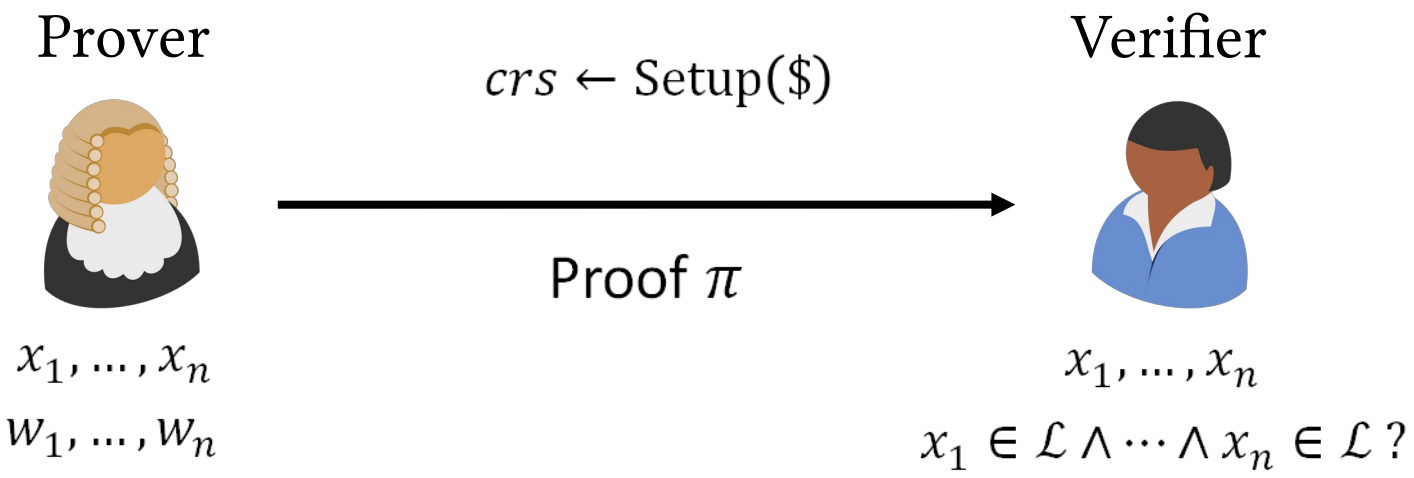
- We overcome this limitation – achieve adaptive multi-hop

Techniques

Solution 1

- BARG-based (from LWE)

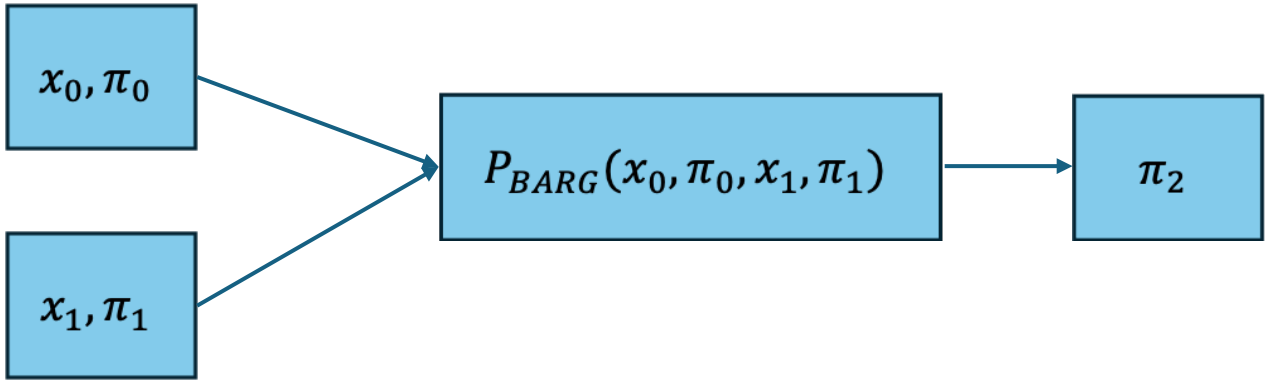
Batch Arguments (BARGs) [BHK17, CJJ21]



Efficiency: $|crs|, |\pi| = poly(\lambda, |w_i|)$

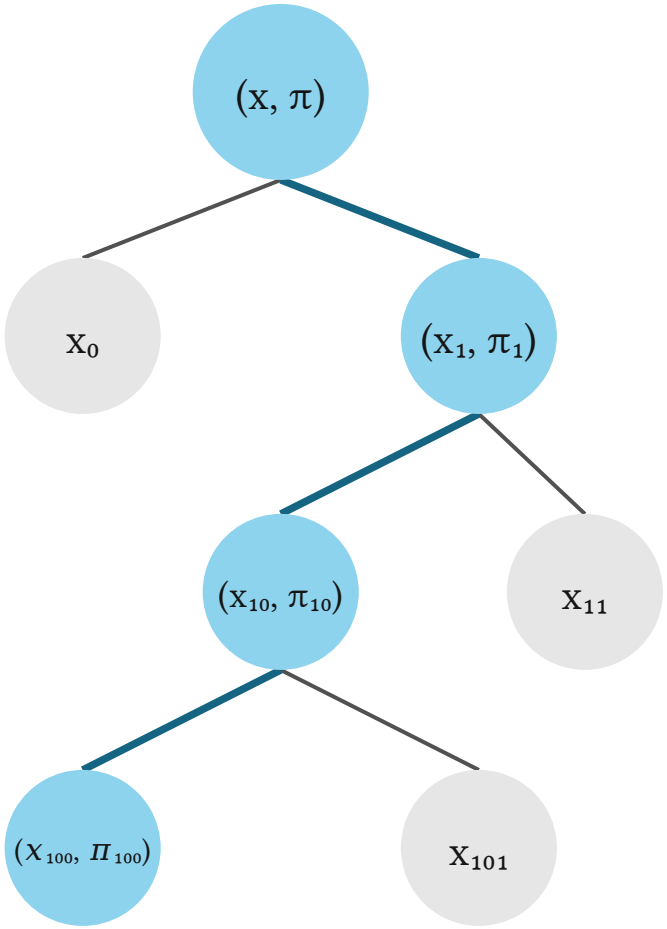
Template Construction From BARG

- Let (x_0, π_0) and (x_1, π_1) be two statements with proofs
- Prove with BARG the NP statements:
  - For every i, there exists accepting π_i for x_i
- Output of PM: $\pi_2' = (x_0, x_1), \pi_2$



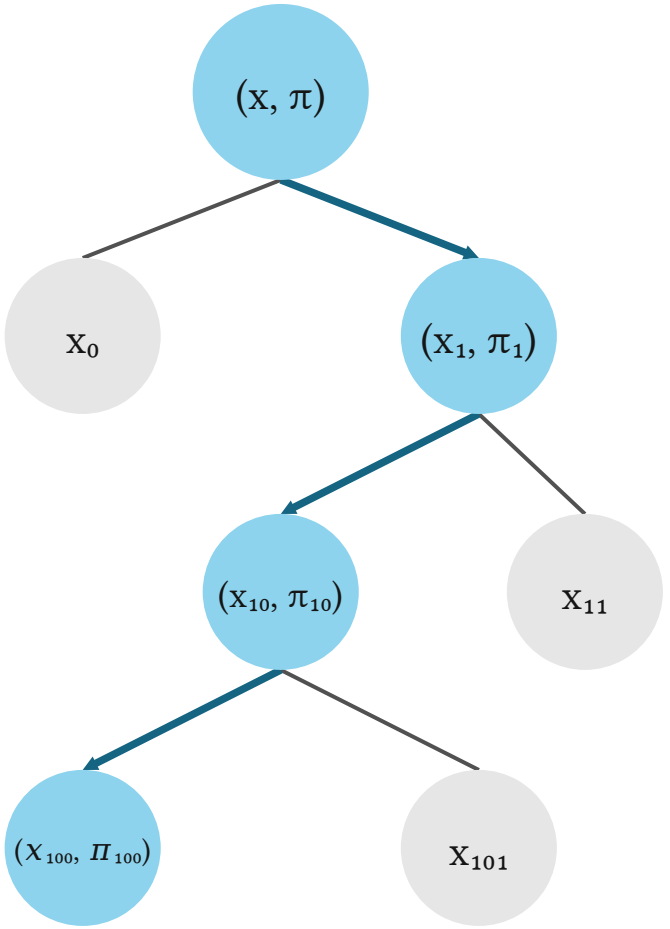
Random-Path Extraction [PP26]

- x_0, x_1 are in the proof π
- Idea: sample a branch at random and extract
  - Use BARG with functional extraction
- "Win" if the sampled **leaf** is false statement with accepting proof
- Security loss scales with $\frac{1}{\#leaves}$
  - Maximal tree size
- What if the merge tree is super-polynomial?



Trapdoor-Guided Extraction

- Idea: use language decider with the trapdoor τ
  - Pick the first s.t. $x_i \notin \mathcal{L}$
- Trapdoor τ encoded in CRS
- Appears only in the reduction



Efficiency

- Required BARG efficiency?
  - Need rate-1 BARG:

$$|\pi_{BARG}| = w + poly(\lambda)$$

- Proof grows with the merge depth d
  - Can make it grow with only the number of hops!

Thank you